

Casualty Care & Transport Documentation System

Offline-first capture of patient identity, injuries, vitals and treatment, with FHIR transfer to the hospital EHR at handover. Merged reference: as-built prototype plus production target architecture.



Document v0.3 – Merged Scope As-built + target architecture

Audience Engineering · Clinical · Compliance

△ Prototype reference – not a medical device; not for clinical use until validated & certified.

PART I · AS BUILT (CURRENT PROTOTYPE)

- | | | | |
|----|------------------------|----|-----------------------|
| 01 | Purpose & context | 02 | Goals & principles |
| 03 | Technology choices | 04 | System structure |
| 05 | Domain model | 06 | Persistence |
| 07 | Runtime data flow | 08 | FHIR interoperability |
| 09 | Offline & PWA strategy | | |

PART II · TARGET ARCHITECTURE (PRODUCTION)

- | | | | |
|----|------------------------|----|-------------------------|
| 10 | Three-zone topology | 11 | Handover data flow |
| 12 | Offline-first & sync | 13 | Deployment topology |
| 14 | Security & privacy | 15 | Regulatory & compliance |
| 16 | Non-functional targets | | |

PART III · FORWARD

- | | | | |
|----|-------------------------|----|--------------------|
| 17 | Roadmap & extensibility | 18 | Build, run, deploy |
|----|-------------------------|----|--------------------|

PART I

As Built — current prototype

The system exactly as it exists in this repository today: a client-only, offline-first PWA — capture and FHIR export run entirely on the device, with no backend required. PHI can be encrypted at rest (opt-in photo vault + passphrase-encrypted backups); end-user auth is still target-only.

01 Purpose & context

TRIAGE-LINK is a digital replacement for the paper triage tag used by emergency responders at accident scenes and mass-casualty incidents. A single responder, often with no network connectivity, captures a patient's identity, injuries, vital signs and treatments at the point of care, then hands that record to a receiving hospital as an HL7 FHIR R4 bundle.

ACTOR	ROLE
Field responder (paramedic, medic, first-aider)	Captures the casualty record on a phone/tablet, usually offline
Receiving facility (hospital EHR)	Imports the exported FHIR bundle at handover

Driving requirements

- ▶ **Works with no connectivity.** The scene may have no signal; the app must be fully functional offline and lose no data.
- ▶ **Runs anywhere, installs like an app.** Responders carry heterogeneous devices; one codebase must run on all of them.
- ▶ **Interoperates with hospital systems.** Handover output must be a format hospital EHRs already understand.
- ▶ **Fast, low-friction capture.** Tap-to-mark injuries; auto-save; no modal "save" step.

02 Goals & principles

PRINCIPLE	HOW IT SHOWS UP IN THE CODE
Offline-first	All state lives on-device in IndexedDB; no backend is required for the core workflow
Portability first	Delivered as a PWA — one static bundle runs on any modern browser and installs to the home screen
Framework-free core	<code>src/domain</code> and <code>src/fhir</code> are plain TypeScript with zero React/Dexie imports — reusable by a future React Native client or backend unchanged
Single source of truth	The <code>CasualtyRecord</code> type in <code>src/domain/types.ts</code> is the canonical model every other layer maps to/from
Standards over bespoke formats	Interop is via HL7 FHIR R4 with LOINC-coded vitals, not a proprietary schema
Local-only by default	No PHI leaves the device unless the user explicitly exports a bundle

03 Technology choices

CONCERN	CHOICE	RATIONALE
Client framework	React 18 + TypeScript	Component model for the capture UI; static typing across the domain
Build / dev server	Vite 5	Fast dev server, simple static production build
Offline storage	IndexedDB via Dexie 4	Durable, async, on-device store with a typed table API; no backend needed
Offline shell	vite-plugin-pwa (Workbox)	Service worker precaches the app so it loads with no network
Interop	HL7 FHIR R4	The de-facto hospital EHR exchange standard

There are intentionally **no runtime dependencies** beyond React and Dexie — the domain and mapping logic are hand-written TypeScript.

04 System structure

The app is organised in layers, from a framework-free core outward to the UI. Dependencies point **inward only**: the UI depends on the domain, never the reverse.

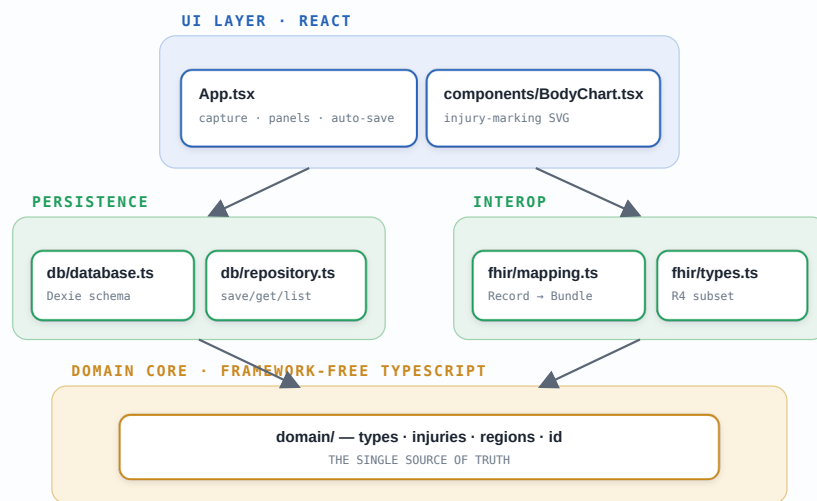


FIG 1 – As-built layered structure. Dependencies point inward only: the UI depends on the domain, never the reverse, so the core is reusable by other clients.

Directory map

```
src/
domain/      Framework-free model – reusable everywhere
  types.ts   CasualtyRecord and all sub-types; triage labels/colours
  injuries.ts Catalog of injury types (key, label, colour) + lookups
  regions.ts Body-region hit-testing over the SVG coordinate space
  id.ts      Case-ID and local-ID generation
fhir/       HL7 FHIR R4 mapping – framework-free
  types.ts   Minimal FHIR resource/bundle types
  mapping.ts CasualtyRecord -> FHIR Bundle
db/         On-device persistence
  database.ts Dexie schema (records / ops / meta / photos tables)
  repository.ts save / get / list / remove / clear (+ op-log journaling)
  crypto.ts   WebCrypto AES-256-GCM + PBKDF2 primitives
  vault.ts    opt-in photo vault (encrypt photos at rest + lock)
  backup.ts   plain & passphrase-encrypted backup / restore
components/
  BodyChart.tsx Interactive anterior/posterior injury-marking SVG
App.tsx     Capture UI; wires domain + db + fhir together; auto-save
main.tsx    React entry point
styles.css  Application styling
```

05 Domain model

The entire clinical record for one patient is a single `CasualtyRecord` (`src/domain/types.ts`):

```
CasualtyRecord
├─ id                stable case ID (also seeded as the MRN)
├─ tombstone         Tombstone  identity: name, dob, sex, mrn, bloodType, nextOfKin...
├─ incident          Incident    injuryTime, mechanism, location, triage category
├─ injuries          Injury[]    each: view, x/y, region, type, severity, notes
├─ vitals            VitalSign[] each: takenAt + hr/bp/rr/spo2/gcs/pain
├─ treatments        Treatment[] each: performedAt, type, detail, place, provider
├─ handover          Handover?   at, clinician, facility (null until handed over)
├─ createdAt         number
└─ updatedAt         number
```

- ▶ **One record = one unit of sync.** The Dexie store keys on `id`; the roadmap treats a record as the atomic unit a sync/op-log layer reconciles.
- ▶ **"Tombstone" = stable identity layer**, deliberately separated from episode-specific `incident` data — mirroring how hospitals separate a Patient from an Encounter.
- ▶ **Triage** uses the standard START scheme (immediate/delayed/minor/deceased), with labels and colours centralised in `TRIAGE_LABELS` / `TRIAGE_COLORS`.

5.1 Body-region resolution

`domain/regions.ts` divides the silhouette's SVG user space (`viewBox 0 0 220 440`) into rectangular hit-test zones. On a tap, `regionAt(x, y, view)` finds the containing region box, applies **anatomical sidedness** (anterior view: image-left is the patient's right; posterior flips it), and falls back to a vertical-band heuristic outside any box. This rectangular model is a placeholder; the roadmap replaces it with a precise anatomical SVG (named bones, burn TBSA).

06 Persistence

`src/db/` provides durable, offline on-device storage. `database.ts` declares a Dexie database named `triage-link` with four tables: `records` (indexed on `id` and `updatedAt`), `ops` (the append-only sync op-log), `meta` (sync bookkeeping + the vault config), and `photos` (out-of-line wound-photo blobs). `repository.ts` is a thin repository exposing `save`, `get`, `list`, `remove`, `clear` — `save()` stamps `updatedAt`, dehydrates embedded photos to `idb:<id>` references, journals the field/item diff as immutable ops, and writes records + ops + photos in one Dexie transaction; `list()` stays lightweight (newest-first, no photo rehydration).

Because IndexedDB is local to the device, the core workflow needs **no server**. The repository is intentionally thin so the sync layer can wrap it with an operation log without the UI changing.

6.1 · At-rest encryption (as built)

PHI is most exposed in the **exported backup file** (it leaves the device) and the **wound photos** (the heaviest, most identifiable PHI). Both can be encrypted with **AES-256-GCM**, the key derived from a user passphrase via **PBKDF2-SHA-256 (210,000 iterations)** — WebCrypto only, no dependency (`src/db/crypto.ts`).

- **Encrypted backups** (`src/db/backup.ts`) — alongside the plain JSON export, a backup can be written as a `pbkdf2-aesgcm-v1` envelope carrying only ciphertext + the KDF salt. Restore auto-detects encrypted vs plain and prompts for the passphrase when needed.
- **Photo vault** (`src/db/vault.ts` , opt-in, default off) — when enabled, photo blobs are encrypted at rest. `enableVault()` derives a key, encrypts every existing photo, and persists a passphrase *verifier* so a wrong passphrase is rejected without touching a photo. `photos.ts` then transparently encrypts on save and decrypts on load using the in-memory key. The key lives **only in memory** while unlocked; a full-screen **lock screen** gates the UI and a 5-minute **auto-lock** drops it on inactivity. `disableVault()` decrypts everything back. Only the photo bytes are encrypted (the indexed keys stay clear, so no migration is needed); record text fields remain in the clear today (§14). A lost passphrase is unrecoverable — the encrypted backup is the recovery path.

07 Runtime data flow

7.1 Capture & auto-save

```
User edits a field / taps the body chart
→ App mutator builds the next immutable CasualtyRecord
→ setRecord(next)           React re-renders immediately
→ debounce 400 ms         coalesces rapid edits
    (after quiet period)
→ recordRepo.save(next)     writes to IndexedDB
→ recordRepo.list()         refreshes "Saved casualties"
```

State updates are immutable (each mutator spreads a new record), keeping React rendering predictable. The 400 ms debounce means rapid typing produces one write per quiet period rather than one per keystroke.

7.2 Injury placement

```
Tap on BodyChart SVG
→ toUserSpace()   screen coords → SVG user space via getScreenCTM().inverse()
→ bounds check    ignore taps outside the body box
→ regionAt(x,y,view) resolve anatomical region
→ onPlace()       App appends a new Injury (active type, default severity)
→ marker rendered; injury selected for inline severity/notes editing
```

7.3 FHIR export at handover

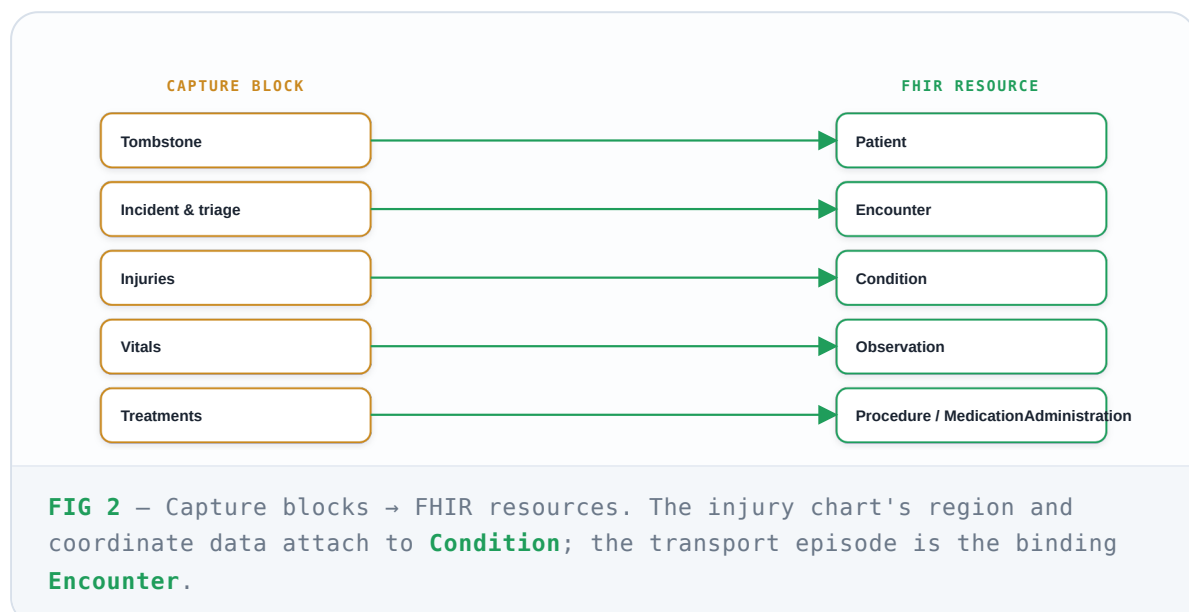
```
"Export FHIR ↓"
→ toFhirBundle(record)           pure function, no I/O
→ JSON.stringify(bundle, null, 2)
→ Blob (application/fhir+json)
→ object URL → anchor click → download "<case-id>-fhir-bundle.json"
```

Export is a pure, client-side transform; nothing is transmitted.

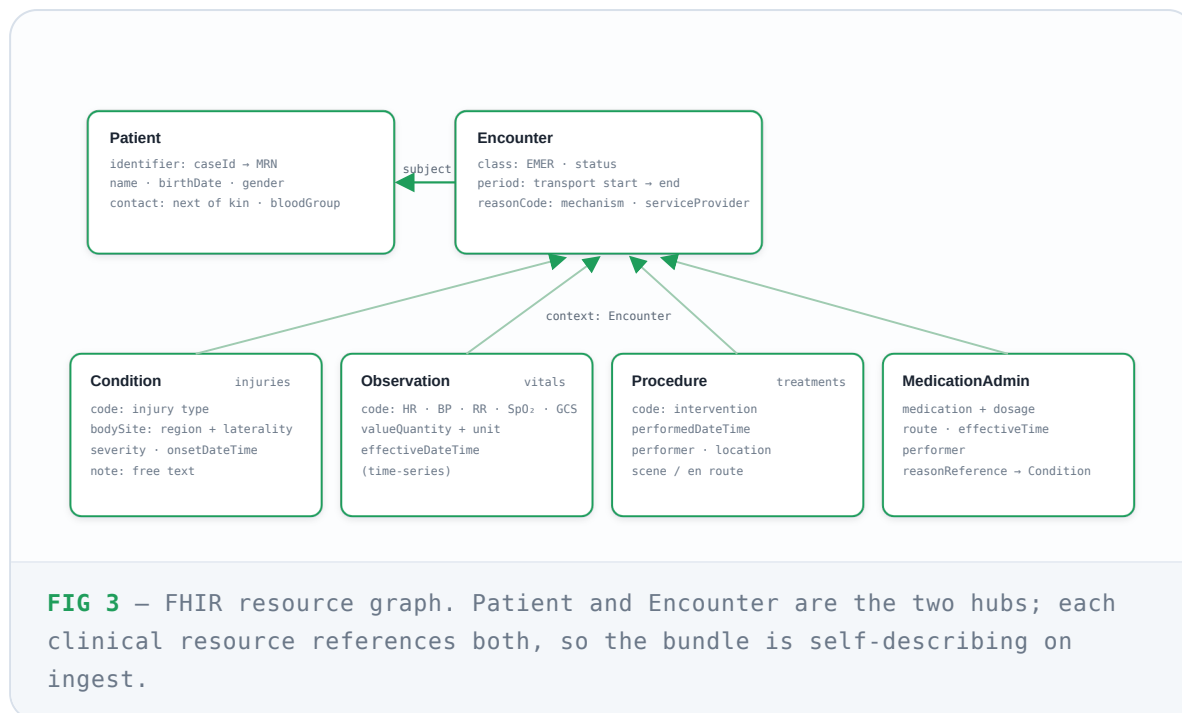
08 FHIR interoperability

`src/fhir/mapping.ts` translates the internal `CasualtyRecord` into a FHIR R4 **Bundle** (`type: collection`). The mapping mirrors clinical semantics:

DOMAIN CONCEPT	FHIR RESOURCE	NOTES
Tombstone (identity)	<code>Patient</code>	name, gender, birthDate, contact; identifier system <code>urn:trriage-link:case</code>
Incident / transport episode	<code>Encounter</code>	<code>class = EMER</code> ; <code>in-progress</code> until handover; mechanism → <code>reasonCode</code>
Each injury	<code>Condition</code>	category <code>injury</code> ; <code>bodySite</code> = region + view; severity; notes
Each vital sign	<code>Observation</code>	category <code>vital-signs</code> ; LOINC- coded (HR 8867-4, SpO ₂ 59408- 5, BP 85354-9); value + unit
Treatment (non- drug)	<code>Procedure</code>	status <code>completed</code> ; performer = provider; detail + place in note
Treatment (medication)	<code>MedicationAdministration</code>	chosen when the intervention type matches <code>/medication/i</code>



Expanded, the resources form a small graph. **Patient** is the identity anchor and **Encounter** the transport episode that binds everything; every clinical resource references both, so the bundle is self-describing on ingest.



i Scope note. The bundle is a collection, not a transaction, and `fhir/types.ts` is a pragmatic R4 subset rather than the full schema. A production integration would validate against a FHIR server and likely use profiles (US Core / IPS).

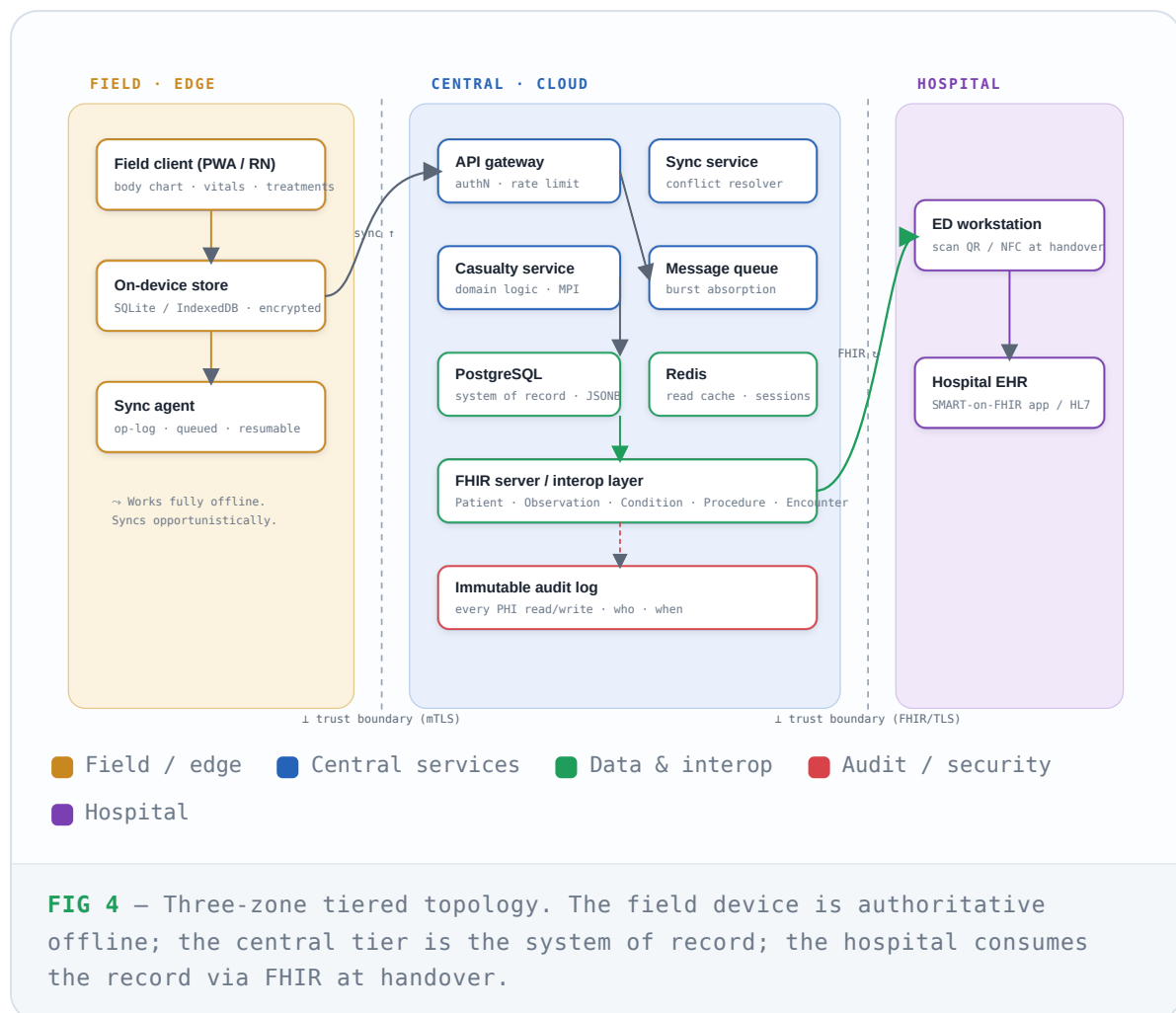
09 Offline & PWA strategy

- **App-shell caching.** vite-plugin-pwa (Workbox) generates a service worker that precaches the built assets, so after the first load the app opens with no network.
- **Data offline.** All records persist in IndexedDB; there is no network read path in the core workflow, so being offline is the *normal* operating mode, not a degraded one.
- **Static deployment.** `npm run build` type-checks and emits a static `/dist` bundle deployable to any static host or CDN. No server-side runtime is involved.

The prototype is the field client of a larger system. This part describes the production topology it is designed to grow into — central services, conflict-aware sync, hospital integration, and the security/regulatory envelope. None of this is implemented yet; it keeps current decisions aligned with where the system is headed.

10 Three-zone topology

Three trust zones — field/edge device, central cloud, and hospital — across two trust boundaries. The field client owns the record offline and syncs to the central system of record when a link returns; the hospital receives it over HL7 FHIR at handover.



11 Handover data flow

The record moves from scene to hospital. The handover scan (QR/NFC of the case ID) pulls it into the receiving facility, where the master patient index reconciles the field case ID to a real MRN. Steps 1-2 require no connectivity.

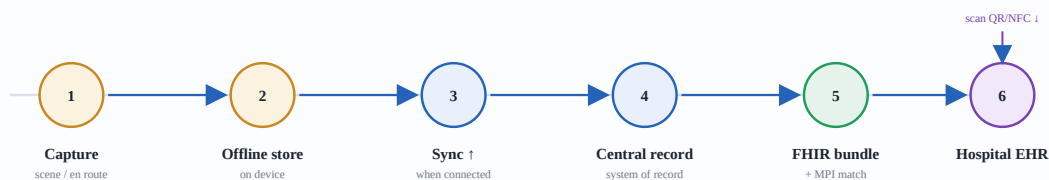


FIG 5 – Capture → offline persistence → opportunistic sync → system of record → FHIR exchange → EHR. Steps 1-2 require no connectivity.

12 Offline-first & sync

The hardest part of the system. Plain *last-write-wins* is unsafe for medical data — two responders editing the same casualty could silently erase a treatment. The model is an append-only change log per device, merged centrally with deterministic conflict resolution and a full audit trail.

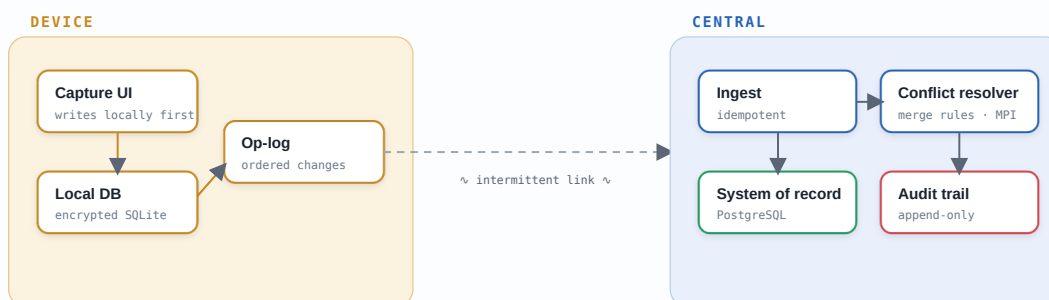
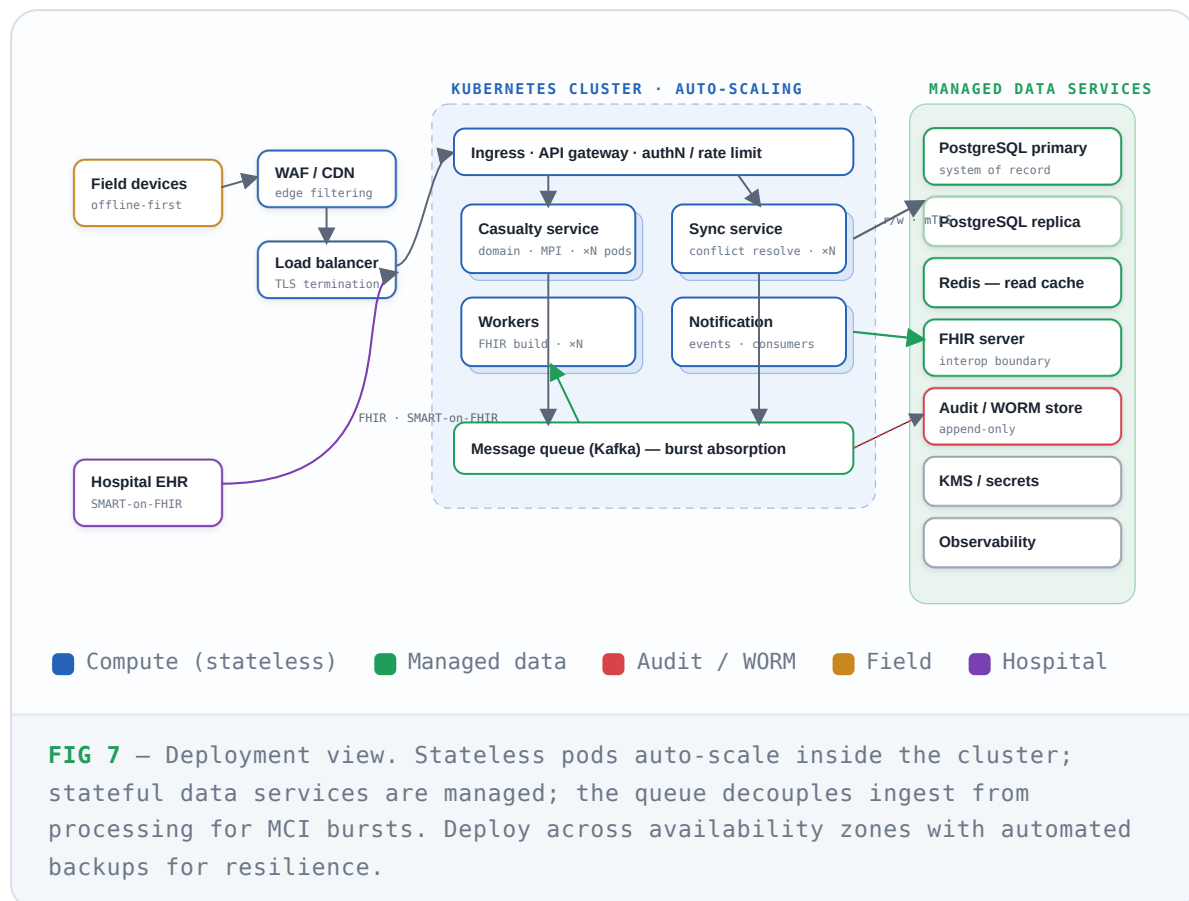


FIG 6 – Local-first writes flow through an ordered op-log; the central tier ingests idempotently, resolves conflicts deterministically, and records every merge.

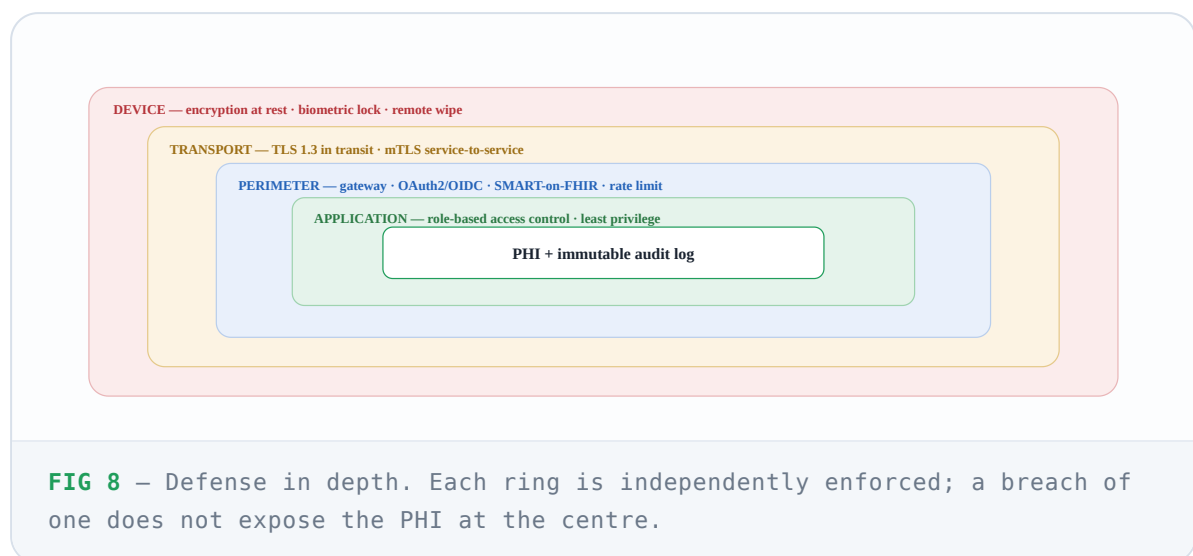
13 Deployment topology

Stateless services on a managed Kubernetes cluster behind a load balancer and WAF scale out horizontally; data services are managed and stateful. During a mass-casualty incident the queue absorbs the write burst while pods auto-scale, then smooths the reconnection storm. The hospital reaches the FHIR endpoint through the same secured edge via SMART-on-FHIR.



14 Security & privacy

Status: partially implemented. Hardened so far: at-rest encryption of wound photos (opt-in vault) and exported backups, both AES-256-GCM/PBKDF2 (§6.1); a passphrase lock screen with auto-lock; backend API auth (bearer token, constant-time compare), rate-limiting, CORS allowlist and `helmet` headers on the sync service; a client Content-Security-Policy and security headers; and CI security scanning (CodeQL, secret scan, `npm audit`). Still open: end-user auth on the PWA, encryption of record text fields, RBAC, and immutable audit logging. A real deployment applies defense-in-depth across every layer rather than a perimeter alone.



DOMAIN	REQUIREMENT	
Encryption	TLS 1.3 in transit; AES-256 at rest on device, DB, backups; mTLS between services. <i>Partly built:</i> AES-256-GCM at rest for photos (opt-in vault) and encrypted backups (§6.1)	Must
Device security	Full-disk encryption, biometric lock, MDM, remote wipe for lost devices	Must
Authentication	OAuth2/OIDC for users; SMART-on-FHIR for EHR integration; MFA for privileged roles	Must
Authorisation	RBAC with least privilege; field / dispatch / clinician / admin scoped separately	Must

DOMAIN	REQUIREMENT	
Audit logging	Immutable, append-only log of every PHI create/read/update/delete	Must
Integrity at handover	Signing / provenance on the exported bundle	Should
Key management	Centralised KMS/HSM; rotation; no secrets in source or images	Should
Pen testing	Independent security testing and disclosure process before go-live	Recommend

15 Regulatory & compliance

Two questions shape the whole lifecycle: **is this a regulated medical device?** (FDA SaMD / EU MDR if it influences clinical decisions) and **which privacy regime applies?** (depends on where patients and data live).

AREA	WHAT IT REQUIRES	APPLIES WHEN
HIPAA (US)	Privacy & Security Rules; Business Associate Agreements; breach notification	US patients / providers
GDPR (EU/UK)	Lawful basis, data-subject rights, DPIA, residency, 72-hour breach notice	EU/UK data subjects
SaMD / EU MDR	Design controls, ISO 14971 risk management, clinical evaluation, validated SDLC	Software drives clinical decisions
IEC 62304	Medical-device software lifecycle processes	Regulated software build
ISO 27001 / SOC 2	Information-security management system; independent attestation	Enterprise / hospital procurement
HL7 FHIR	Interoperability conformance with certified EHRs	EHR integration (always)
Data residency & retention	Store PHI in approved jurisdictions; defined retention & destruction	Jurisdiction-dependent

§ **Decide device classification early.** Whether this is a documentation tool or a clinical-decision device changes cost, timeline and process dramatically — far cheaper to design for the right class than to retrofit.

16 Non-functional targets

ATTRIBUTE	TARGET
Offline capability	100% of capture functions usable with zero connectivity
Sync latency	Record reaches central tier within seconds of connectivity returning
Availability	≥ 99.9% for central services; handover path prioritised
MCI burst	Sustain many concurrent responders without capture-side degradation
Handover time	Scan-to-EHR transfer in seconds, not minutes
Data integrity	No silent data loss; all conflicts resolved deterministically & audited

PART III

Forward

Roadmap, build commands, and the design decisions that tie the two parts together.

17 Roadmap & extensibility

Because the domain and FHIR layers are framework-free, several roadmap items can be added without touching the UI:

- ▶ **Conflict-aware sync.** Wrap `recordRepo` with an operation log and reconcile records against a central service (§12).
- ▶ **Anatomical body chart.** Replace the rectangular `regions.ts` zones with a precise anatomical SVG (burn TBSA, named bones) — `regionAt()` is the single seam to swap.
- ▶ **Handover scanning.** NFC/QR handover plus master-patient-index reconciliation.
- ▶ **Security hardening.** Auth (OAuth2/OIDC, SMART-on-FHIR), audit logging, and extending at-rest encryption to record text fields — photos & backups already done (§6.1, §14).
- ▶ **Reuse on other clients.** The `domain` + `fhir` modules could back a React Native app or a Node sync service unchanged.

18 Build, run, deploy

```
npm install
npm run dev      # Vite dev server at http://localhost:5173
npm run typecheck # tsc --noEmit
npm run build    # type-check + production build to /dist
npm run preview  # serve the production build locally
```

`/dist` is a self-contained static bundle; the service worker makes it work offline after the first load.

19 Key design decisions

- ▶ **PWA over native** — maximum portability from one codebase; installable; offline-capable; static deployment.
- ▶ **Framework-free domain & FHIR core** — reusable across future clients and services; the `CasualtyRecord` is the single source of truth.
- ▶ **IndexedDB/Dexie, record-as-sync-unit** — durable offline storage with a clean seam for a future op-log sync layer.
- ▶ **FHIR R4 for interop** — speak the hospital's language at handover rather than inventing a format.
- ▶ **Immutable state + debounced auto-save** — predictable rendering and low-friction, loss-resistant capture.

TRIAGE-LINK · Technical Architecture · v0.3 Merged — Prototype reference, not for clinical use.
Part I reflects the current codebase; Part II is the proposed production target.
Device classification, privacy regime and data residency are prerequisites to build.